

Übung2

November 12, 2025

1 Übung 1: Funktionen

1.0.1 Aufgabe: Schreibe zwei Funktion die zwei gleichlange Listen mit Zahlen elementweise addiert und subtrahiert

Hinweise: - Die Aufgabe benötigt **for-loops** - Was passiert wenn die Listen nicht gleich lang sind?
- Resultate müssen ausgegeben werden mit **return** - Eine leere Liste kann mit [] erstellt werden, neue Werte können mit **.append()** hinzugefügt werden

```
[ ]: def addition():  
    # Addiere die zwei Listen hier  
  
def subtraktion():  
    # Subtrahiere die zwei Listen hier  
  
list1 = [4.3, 2.4, 5.3, 1.2, 4.5]  
list2 = [9.4, 9.2, 1.4, 2.2, 7.8]  
  
add_list = addition()  
sub_list = subtraktion()
```

1.0.2 Aufgabe: Der folgende Code kann durch eine Funktion stark gekürzt werden, indem wiederholende Abschnitte durch einen For-loop mit einer Funktion erledigt werden.

Hinweis: - Viele gleiche Werte können in **Listen** zusammengefasst werden

```
[ ]: # Input Daten  
m1 = 10    # Masse Probe 1  
m2 = 3     # Masse Probe 2  
m3 = 12    # Masse Probe 3  
m4 = 15    # Masse Probe 4  
  
V1 = 102   # Volumen Lösung 1  
V2 = 99    # Volumen Lösung 2  
V3 = 109   # Volumen Lösung 3  
V4 = 103   # Volumen Lösung 4
```

```

molmasse = 99    # g / mol

# Konzentrationsberechnung
def konzentration(gewicht, molmasse, volumen):
    # Berechnet die Konzentration in mol/l
    # gewicht: Stoffmenge in Gramm
    # molmasse: Molekulare Masse in g / mol
    # volumen: Lösungsmittel in ml
    """
    Berechne die Konzentration in mol / l hier
    """

# Berechnung der Stoffmengen
n1 = m1 / molmasse
n2 = m2 / molmasse
n3 = m3 / molmasse
n4 = m4 / molmasse

# Volumen in Litern
V1 = V1 * 1000
V2 = V2 * 1000
V3 = V3 * 1000
V4 = V4 * 1000

# Berechnung der Konzentrationen
c1 = n1 / V1
c2 = n2 / V2
c3 = n3 / V3
c4 = n4 / V4

# Output
print("Konzentration 1 beträgt", c1)
print("Konzentration 2 beträgt", c2)
print("Konzentration 3 beträgt", c3)
print("Konzentration 4 beträgt", c4)

```

2 Übung 2: Indizierung von Arrays

2.0.1 Aufgabe: Lass dir die jeweiligen gefragten Werte aus dem Array angeben.

```

[ ]: import numpy as np

my_array = np.array([
    [ 1,  2,  3,  4,  5],
    [ 6,  7,  8,  9, 10],
    [11, 12, 13, 14, 15],

```

```

    [16, 17, 18, 19, 20]
])

""" TODO 1: Ausgabe des Elements mit Wert 3 """
result_1 =
print(result_1)

""" TODO 2: Ausgabe des Elements mit Wert 19 """
result_2 =
print(result_2)

""" TODO 3: Ausgabe der ersten Spalte """
result_3 =
print(result_3)

"""
TODO 4: Ausgabe der Sub-Matrix
    [[14, 15],
     [19, 20]]
"""
result_4 =
print(result_4)

""" TODO Bonus: Ausgabe aller Werte größer gleich 16 und kleiner 19 """
print()

```

3 Übung 3: Numerische Integration

Das Integral über eine Gaussfunktion ist gleich der Quadratwurzel der Kreiszahl π .

$$\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi}$$

Integriere die Gaussfunktion numerisch und vergleiche das Ergebnis mit der analytischen Lösung. Benutze dazu die Funktion `numpy.trapz()` ([Dokumentation](#)). Bei einer numerischen Integration kann das Integral über den Bereich $(-\infty, +\infty)$ durch ein Integral über ein endliches Intervall $[a, b]$ angenähert werden. Hilfreiche `numpy` Funktionen/Konstanten für diese Aufgabe: `numpy.pi`,

```

[ ]: """TODO: importiere das numpy modul
      mit alias np
"""
import

"""TODO: definiere eine Funktion, die für ein

```

```

        gegebenes x eine Gaussfunktion zurück
        gibt
"""
def gauss_function(x):
    return ____

"""TODO: erstelle einen 1D-Array"""
x = ____
y = gauss_function(x)

"""TODO: numerische Integration"""
numeric_integral = ____

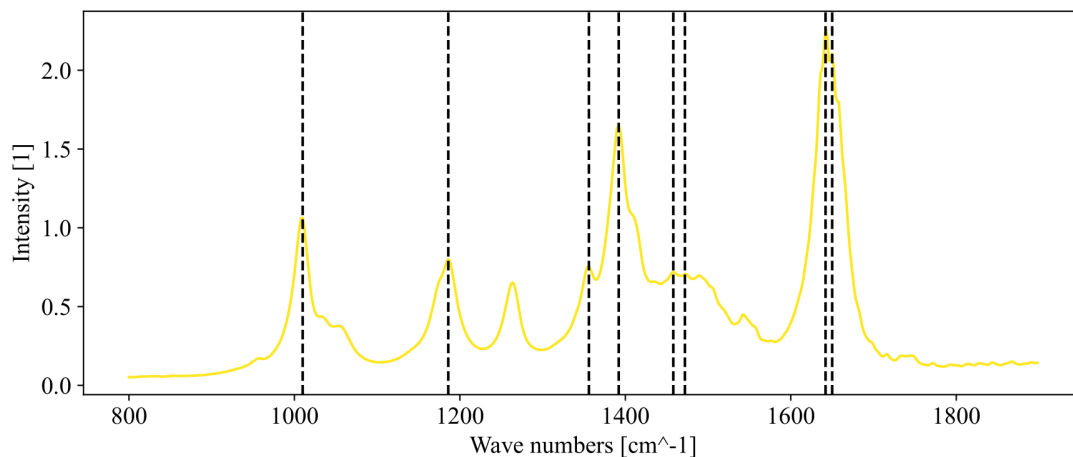
"""TODO: analytische Lösung"""
correct = ____

print(f"Numerisches Integral: {numeric_integral}")
print(f"Analytische Lösung: {correct}")
print(f"Differenz: {numeric_integral - correct}")

```

4 Spektrum analysieren

Für die Übung ist ein IR-Spektrum von Dimethylacetamid (DMAc) gegeben.



###

Übung 4: Daten Laden und auswerten Nutze `numpy.loadtxt()` um das Spektrum aus `dmac.csv` zu laden. Finde anschließend das Absorptionsmaximum und lass dir die Wellenzahl des Maximums ausgeben.

```

[ ]: import numpy as np
      dmac_spectrum = np.loadtxt().T

      print()

```

4.0.1 Finde die Positionen der anderen markierten Peaks in der Abbildung

Lokale Absorptionsmaxima, die keine globalen Maxima sind, können mit Funktionen, die das Maximum aus einer Liste oder einem Array suchen schwer zu finden sein. Da wir aber wissen **wo** die verschiedenen Maxima auftauchen, können wir den Suchbereich eingrenzen um die gewünschten Peaks zu erhalten. Wir möchten die exakten Positionen von 4 Peaks finden:

- Zwischen 900 cm^{-1} und 1100 cm^{-1}
- Zwischen 1100 cm^{-1} und 1300 cm^{-1}
- Zwischen 1300 cm^{-1} und 1500 cm^{-1}
- Zwischen 1600 cm^{-1} und 1800 cm^{-1}

Die Positionen der Peaks kann bestimmt werden, indem zunächst das Spektrum mit der Funktion `mask_spectrum` auf einen Teilbereich eingegrenzt wird, um danach innerhalb dieses Teilbereichs mit `np.argmax()` das Maximum zu finden.

4.0.2 Hinweise:

- List comprehensions können nützlich und effizient sein:

```
some_list = [element for element in Liste if Bedingung]
```

- Über mehrere Listen kann mit der `zip` Funktion geloopt werden:

```
for el1, el2 in zip(list1, list2):
```

```
[ ]: def mask_spectrum(x, y, x_min, x_max):  
    """  
    Entfernt den Teil des Spektrums der nicht zwischen x_min und x_max liegt  
    x: Liste der Wellenzahlen  
    y: Liste der Intensitäten  
    x_min: Kleinster X-Wert im Ausschnitt des Spektrums  
    x_max: Größter x-Wert im Ausschnitt des Spektrums  
    Output:  
    x_masked: Wellenzahlen im Bereich x_min bis x_max  
    y_masked: Absorption im Bereich x_min bis x_max  
    """  
  
    return x_masked, y_masked  
  
    '''TODO: Spektrum wie in der vorigen Übung laden und x- bzw. y-Werte_  
    ↪definieren'''  
  
x =  
y =  
  
peak_x_min = [900, 1100, 1300, 1600]  
peak_x_max = [1100, 1300, 1500, 1800]  
  
for i in :  
    print(f"Das Maximum von Peak {i} liegt bei {} cm-1")
```